

A developer who digs into root causes and enjoys solving difficult problems

The problems I encountered, the experiments and results, and the decisions that shaped each implementation.

Jaehyeong Kim

AI Application Developer · Backend Engineer

kkuldangi2@gmail.com · martinel2.github.io

I owned the whole AI side of an LLM workspace, designing its RAG, agent and evaluation layers and building and verifying them with AI coding tools.

I weigh accuracy, cost and response time against each other, confirm the trade-off with an evaluation, and then choose the implementation that fits the service.

RAG evidence selection: coverage 66.25% → 91.25%, false evidence 20 → 0

Built a golden set and ran a blinded offline comparison with a matched candidate pool and two independent judge models, then replaced the evidence-selection stage. Tuning candidate count and parallel batches cut median selection time from 9.090s to 2.251s.

Agent execution environment: user-defined skills behind approval, edit quality 82.5% → 91.2% (114 drafts)

Split responsibility so code enforces permissions and prompt-injection checks, the model drafts instructions and edits, and the user publishes and approves. Nothing changes in a document before the user approves it.

Data pipelines: PDF conversion pass rate 45.17% → 89.89%, drug conversion \$200 estimated → \$11.18 spent

Separated processing paths for body text, tables and equations so documents become usable for retrieval and answers, and processed about 44,000 drug records within budget through deduplication and batch conversion.

Contents

Short on time? Start with 01, 02 and 04 — the three experiences on the cover: quality evaluation, the agent execution environment, and data pipelines.

FRUITION

01	RAG evidence coverage: 66.25% → 91.25%	p.6
02	Agent execution environment: authoring, validation and approval	p.13
03	Task cancellation - undoing saved changes in reverse	p.19
04	PDF conversion: pass rate 45.17% → 89.89%	p.23
05	Parallel analysis: 73.8% less processing time	p.28
06	Agent loop: 16 PRs merged in 5 days	p.30

PILLTIP

07	Drug data conversion: \$200 estimated → \$11.18 spent	p.37
08	Medication chatbot: health profiles kept internal	p.42

MORE

09	Beyond project development	p.45
	Rhwp · Fixing HWPX save errors	
	APPTIVE · Backend mentoring	
	945-day Baekjoon problem-solving streak	
10	Project reflections	p.46

Fruition

An AI workspace that organizes uploaded documents into an LLM-generated Wiki, answers questions using source evidence through RAG, and edits documents in natural language. It is currently deployed on AWS EKS and undergoing QA.

How it started

Started when a teammate raised a pain point with managing documents, which we refined through design thinking. The goal was to fix the frustration of saving many documents but failing to find them when needed.

GitHub · AI repository ↗

Team and my role

MY ROLE

AI

All AI features: PDF conversion, Wiki generation, RAG retrieval and grounded answers, the document-editing Agent

01 Lead for all AI features

Led design and evaluation across PDF conversion, Wiki generation and organization, retrieval and grounded answers, and the document-editing Agent, with implementation done using AI coding tools.

02 MSA design contribution

Proposed the initial architecture, then finalized service boundaries, data ownership and communication with teammates and mentors.

Backend, 1

The entire Spring-based backend

Frontend & DevOps, 1

The entire frontend and MSA-based deployment on AWS

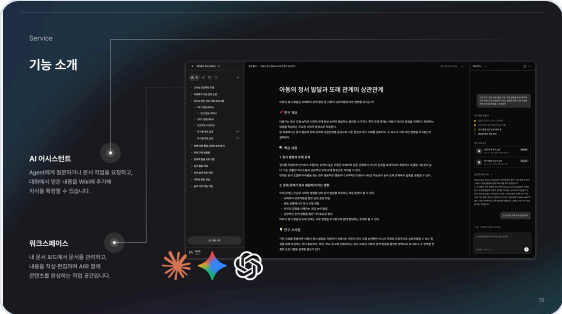
Design

External designer

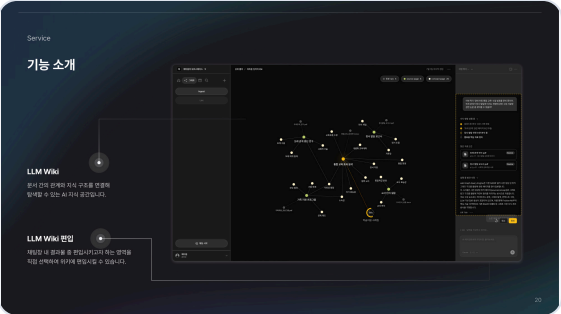
Mentors, 3

Gen AI quality and evaluation / backend / milestones, MSA design and documentation

Features and service screens



Document editing and AI assistant
I led the AI features that interpret requests and generate and validate edits. Teammates implemented the editor and assistant UI.



Wiki generation and retrieval
I led the AI features that organize and connect document knowledge and find evidence for questions. Teammates implemented the graph and search UI.

Architecture

시스템 아키텍처

브라우저는 Spring Boot API를 통해 데이터와 AI Service을 사용합니다.

기술	구현
Next.js	문서 · Wiki · Chat · Agent UI
Spring Boot	인증 · 권한 · 워크스페이스 · 상태
FastAPI	Ingest · Query · Lint · 문서 편집 Agent
PostgreSQL	관계 · 버전 · 작업 상태
MinIO	원본 · Markdown · artifact

Service responsibilities

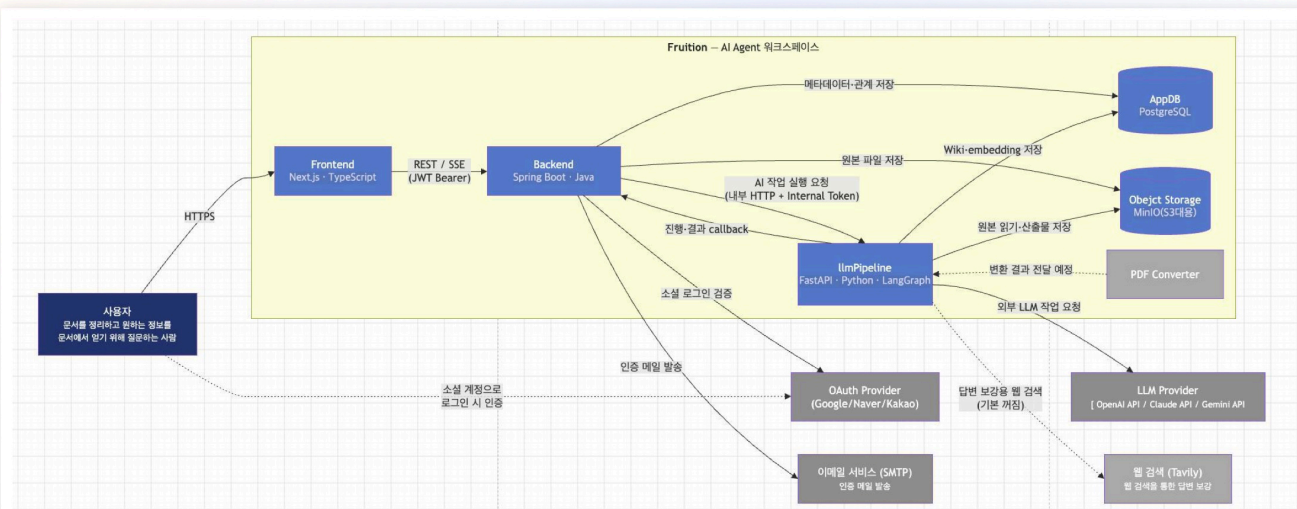
I led all AI features; teammates owned frontend and backend. We finalized MSA with mentors based on my initial design.

System architecture

| C1 전체 시스템 아키텍처 간략화



| C2 Fruition 시스템 일부 구체화



the overall system (top) and Fruition's internal components (bottom). I owned the AI features centred on llmPipeline (FastAPI · Python · LangGraph); teammates owned the frontend, backend and AWS deployment.

RAG evidence coverage: 66.25% → 91.25%, with 75% less time after tuning

Jev is a decision-only model that must pick from a fixed set of options. Applied first to request routing, it scored 81/98 against 77/98 for the existing JSON approach and 63/98 for GPT-5-nano on the same options, with a shorter decision time. I then applied it to RAG evidence selection, raising evidence coverage from 66.25% to 91.25%, and reduced its added latency by tuning candidates and parallel batches.

2026.09

Jev 1.13.0

BGE-M3

BM25

RAG

Quality evaluation

Background

To choose appropriate tasks and answer evidence more reliably, I evaluated Jev first for routing and then expanded it to RAG. Retrieval stays a fixed pipeline that gathers source evidence through semantic and keyword search plus wiki graph traversal, and the answer is presented separately from its evidence, related pages and traversal path. A query agent that plans its own retrieval strategy was deferred. The threshold for adding agents is a single question that demands several retrieval plans and replanning on intermediate results; below it, the fixed pipeline gathers the same evidence in fewer calls and keeps failure points easier to trace.

Evidence coverage
(LLM evaluation on
golden set)

**66.25% →
91.25%**

Jev evidence-selection
time

**Median
9.090s →
2.251s**

False evidence returns
on 20 unanswerable
questions

20 → 0

Experiments and improvements

1 Routing comparison and the decision to try RAG

Problem

Some questions were misclassified as document edits, leading to the wrong action. Task types and additional conditions needed to be selected more accurately and quickly.

Experiment and result

Applied Jev, a choice-based decision model released on September 14, to routing questions, document edits, document creation and folder organization.

Across 98 requests, correct task types and additional conditions totaled 77 for the existing JSON approach, 63 for GPT-5-nano given the same options, and 81 for Jev. Median decision time improved from 8.967s to 0.658s.

I measured API cost on the same 100 questions: about \$0.0408 for Jev against about \$0.1027 for the GPT approach given the same options.

Decision

Jev showed promise on accuracy, speed and cost alike. On that basis I decided to try it on evidence selection, which previous retrieval changes had not solved.

I chose Jev over the alternatives for two reasons. Having GPT pick the evidence cost about 2.5 times as much even with the same options, and took longer, which was too heavy for a step that checks hundreds of candidates per question. Re-ranking by tuning retrieval scores did not help much either: across many hybrid weights and RRF settings, scores stayed at a similar level. Embeddings sometimes failed to separate differences in Korean phrasing, so I judged that fitting one weight to questions from different fields would bring only small gains. Rather than polishing the ranking further, I chose to have a model judge the needed evidence directly from the collected candidates.

Experiments and improvements (continued)

2 Full-corpus inspection: evidence coverage on all 90 answerable questions

Problem

Using the original question for retrieval and increasing semantic-score weight raised top-1 correct retrieval from 50 to 57 out of 110 questions. However, the experiment had some bias and did not establish reliable evidence selection on new questions.

Experiment and result

Tested whether Jev could find evidence missed by retrieval. It inspected all 2,020 evidence units in a test Wiki without retrieval filtering and returned at most 8, matching the existing return limit.

In a semantic evaluation allowing paraphrases, coverage on 90 answerable questions rose from 24/90 (26.7%) to 90/90 (100%). It returned no evidence for all 10 unanswerable questions. Median processing time was 1.511s for the original approach and 5.338s for Jev. Jev processed 26 batches of 80, with up to 4 in parallel.

Full-corpus inspection took 2,600 calls for 100 questions, and the API reported roughly 124.5M input and 6.47M output tokens. I did not confirm the billed amount.

To understand the size of the gap I traced where the existing retrieval dropped the answer. The page holding the reference sentence was in the initial candidate pool for all 90 questions, but 76 fell out at the step that limits inspection to the top 8 pages, 8 at the threshold of 85% of the best in-page score, and the remaining 2 at the final 8-evidence cap and the related-page lookup.

Decision

The existing retrieval was discarding the answer before selection even began, so the gap cannot be read as a difference in selection ability over the same candidates. Full-corpus access found the evidence, but inspecting everything per question would grow calls as documents grew. Rather than set the deployment scope from this 90-question wiki alone, I decided to narrow candidates through retrieval first and let Jev pick the final evidence.

Experiments and improvements (continued)

3 Controlled comparison on 500 candidates: 87.50% coverage

Problem

The full-corpus test needed 26 batches per question. Questions were concentrated in certain technical documents and included document titles. The original search inspected only some pages while Jev saw all evidence, confounding judgment quality with access scope. Both candidate count and comparison conditions needed control.

Experiment and result

Created 100 new questions from 9 papers without reusing earlier questions: 63 answerable from one paper, 17 requiring two papers, and 20 unanswerable. Questions, reference answers and required facts were fixed before runs.

After deduplication, BGE-M3 and BM25 selected up to 500 candidates once; both methods received identical text and metadata. The original selector could inspect every candidate page, and both methods returned at most 8 untruncated evidence passages. Reference answers were excluded from retrieval and Jev inputs. GPT and Gemini graded independently without seeing method names or competing outputs, and run order alternated.

GPT-graded coverage rose from 53/80 (66.25%) to 70/80 (87.50%); Gemini also showed improvement, from 52/80 to 74/80. False evidence returns on unanswerable questions fell from 20 to 1. Median time including candidate preparation was 0.431s for the original selector and 9.090s for Jev, which processed batches sequentially.

Decision

Jev selected better evidence even with matched candidates and return conditions, so I kept retrieval for candidate collection and Jev for final selection. Results on public papers and self-authored questions were not treated as general accuracy. Next I would tune candidate count and batch processing to reduce sequential waiting.

Experiments and improvements (continued)

4 300 candidates and 4 parallel batches to reduce latency

Problem

Jev improved evidence selection, but sequential processing of up to 500 candidates raised median preparation-and-selection time to 9.090s versus 0.431s previously. I needed to reduce waiting without losing accuracy.

Experiment and result

Reduced the candidate pool to the top 300 and ran up to 4 batches in parallel. Time fell to 2.251s, while questions with all required evidence increased from 70 to 73.

Decision

At 2.251s, the tuned approach was still slower than the original 0.431s. I considered fewer candidates and more parallel batches, but rejected further candidate reduction because some required evidence ranked below 250.

I adopted Jev despite the added latency because accuracy was central to the service.

Experiments and improvements (continued)

5 Comparing candidate generators to find the limits of the production setting

Problem

Jev can only choose from what reaches it, so evidence missing from the candidate pool is unreachable. I needed to know how far the production candidate generator preserved the answer-bearing evidence.

Experiment and result

Reusing stored scores, I regenerated rankings for four candidate generators and compared how well each preserved the 97 pre-registered citation groups across the 80 answerable questions: BGE-M3 alone, the production hybrid (0.75 embedding + 0.25 normalized BM25), RRF, and an alternating union.

At 100 candidates, the questions that preserved every citation were 74/80 for BGE-M3 alone, 63/80 for the production hybrid, 63/80 for RRF and 65/80 for the alternating union. MRR@500 over each citation's best rank was 0.1811 for BGE-M3 alone against 0.1245 for the production hybrid, with median ranks of 19 and 40. Beyond 300 candidates the four converged to 79/80 and 78/80.

Decision

The production hybrid weighting was ranking answer-bearing evidence lower than plain BGE-M3. That said, this is a ranking replay over stored scores on an evaluation set I had already reused, so I do not treat it as independent verification, and at the current 300-candidate setting the gap is small enough that I did not change production. I recorded it as a precondition: cutting the candidate count further requires fixing this ranking gap first.



Questions with sufficient evidence: 53/80 → 73/80; total median time 2.251s

Evidence coverage (LLM evaluation on golden set)

66.25% → 91.25%

Original selector 53/80 → tuned Jev 73/80

Jev evidence-selection time

Median 9.090s → 2.251s

500 candidates, sequential → 300 candidates, 4 parallel batches

False evidence returns on 20 unanswerable questions

20 → 0

Original selector → tuned Jev

On a self-authored golden set of 80 answerable questions, the share with evidence supporting every required fact rose from 66.25% to 91.25%. On a separate 20 unanswerable questions, false evidence returns fell from 20 to 0. The improved selector was still slower than the original, but I chose Jev because accuracy mattered more to the service. I also tried grouping equivalent expressions: correct answers stayed similar on 76 questions (71 previously, 72 with Jev), while median decision time fell from 8.162s to 0.254s.

Scope and tradeoffs

The speedup compares Jev before and after its own settings changed. Retrieval gathers candidates and Jev makes the final evidence call, so the selection stage can be swapped on its own. Jev is an external decision model released in September 2026, so I have no long-run operational evidence for it yet, and even after tuning it stays slower than the previous selector. Answers stream to the user, but evidence selection has to finish before generation starts, so the 2.251s remains user-visible waiting and shortening it is still open work. The candidate-generator comparison is a ranking replay over stored scores, and it measures candidate preservation rather than final answer accuracy.

Introducing Jev · [TypeSafe AI ↗](#)

User-defined Agent Skills: designing controlled execution

Designed Agent Skills, implemented with AI coding tools, so users could define and reuse recurring document tasks, such as meeting notes. Connected instruction drafting and checks to user review and publication, followed by separate approval for actual changes.

2026.08 - 2026.09

LangGraph

Agent Skill

Prompt injection defenses

LLM Evaluation

Human-in-the-loop

Context engineering

Rolling conversation memory

LangSmith

LLM observability

Background

Recurring document tasks differ by user, so developers could not build every one in advance. AI-generated edits also reached user review with parts of the request missing or awkward rewording, and code checks on format and scope alone could not catch this.

Original Agent

94 / 114

Code changes, without
evaluator

98 / 114

Final implementation
with evaluator

104 / 114

Decisions and implementation

1 Authoring and publishing user-defined instructions

Problem

Recurring tasks such as meeting notes vary by user; developers cannot prebuild every workflow.

Decision

Let users define and reuse natural-language instructions instead of adding each workflow as a feature. Treat AI-generated instructions as proposals until users review them.

Implementation

Turned natural-language requests into reusable Markdown Skills. Users review and edit drafts; the Skill and version are stored on final publication. A published skill is appended as an immutable version rather than overwritten, with a separate pointer to the version that runs. Which run used which version of the instructions stays traceable, so an earlier definition survives a problematic new version.

Verification

Neither Skills nor temporary drafts persist in the DB before publication. Published instructions can be reused in later tasks.

2 Checking instructions for permission and approval bypasses

Problem

User instructions and reference documents could contain personal data or directions to bypass permissions and approval. Such intent can be phrased many ways, and users can edit drafts after review.

Decision

Use code for explicit risk and permission checks, and an LLM to detect rephrased bypass intent. Do not allow publication based only on an earlier check after users edit the content.

Implementation

Applied rule checks to input and reference documents. The server verified that risky phrases reported by the LLM existed in the source. After checking access and original documents, it sent only necessary structure to the model. Publication reran checks on the final content. Risky phrases were not silently deleted; only user-requested regeneration removed them and rechecked the result.

Verification

Showed blocked input and the problematic location to users, letting them edit it themselves or request regeneration.

Decisions and implementation (continued)

3 Previewing changes and applying them after approval

Problem

Publishing a reviewed Skill does not approve every later document change. Users still need to inspect the target documents and proposed modifications.

Decision

Separate instruction publication from approval of actual changes. Keep task classification in the router and execution in the executor, while letting users inspect what will change.

Implementation

Displayed before/after diffs for edits and a folder-and-document move plan for organization. Code checked output format and edit scope; only approved changes were executed.

Verification

With 6 documents, the plan–approve–execute check made no changes before approval. After approval, it created 3 topic folders and moved the documents.

Decisions and implementation (continued)

4 Deciding the scope of conversation context and edit input

Problem

A long conversation cannot be passed to the model in full, and sending a whole document for an edit drives up cost and latency. I had to decide what to keep and what to cut.

Decision

Keep the last six turns verbatim and compress everything older into a rolling summary. For edits, send only the target line range plus surrounding context, and refuse a range that straddles the boundary of a table or code block.

Implementation

Once recent messages reach six, the rolling summary is regenerated from the previous summary plus those messages; if the summarization call fails the previous summary is kept so the conversation does not break. The router receives the summary and recent messages along with the routing decisions from earlier turns, so it can resolve what a referring expression points back to. Edit input is cut to the target line range with context lines above and below, and ranges crossing a structural boundary are rejected.

Verification

A benchmark measures, across document lengths, how large the transmitted input is relative to the source and the average and p95 processing time.

Decisions and implementation (continued)

5 Tracing LLM runs while bounding sensitive data

Problem

The evaluation loop ran as plain Python, so extraction, evaluation, correction and retry could not be inspected stage by stage. Turning tracing on, however, would send raw tool lookups and personal data to an external tracing tool.

Decision

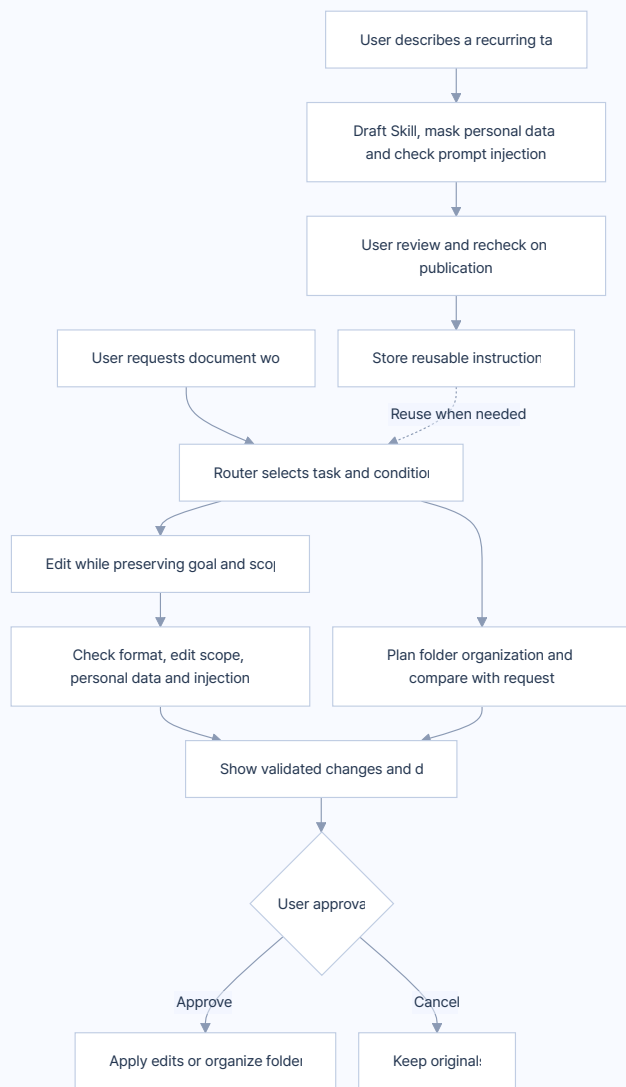
I separated what has to be observable from what must not leave the system. Graphs that decide answer quality get node-level tracing; the agent graph that edits a user's own documents runs with tracing off and keeps raw content out of persisted state.

Implementation

Rebuilt the evaluation loop as a LangGraph graph so answer generation, evaluation and retry each surface as their own node. Even with tracing configured on, a missing key leaves graph execution intact and skips only tracing, which avoids warnings and pointless network retries. The agent graph disables tracing and excludes raw tool lookups from persisted state, and that state is deleted before the run record once retention ends. The prompt log I write myself masks numeric personal data before writing.

Verification

Ran real requests and confirmed that graph nodes and model call spans appear in the trace. Timing experiments run with tracing and prompt logging both disabled, so instrumentation overhead stays out of the measured times.



Reusable tasks and approval workflow; editing passes 82.5% → 91.2% (114 drafts)

Original Agent

94 / 114

82.5%

Code changes, without evaluator

98 / 114

86.0%

Final implementation with evaluator

104 / 114

91.2%

Users can define recurring tasks, publish them as Skills and reuse them. The final instructions are rechecked on publication; actual document changes require a preview and user approval. In a plan–approve–execute check using 6 documents, folder changes occurred only after approval. Editing quality was compared separately on the same 114 drafts from 19 request types. The original implementation passed 94. To reduce errors from re-interpreting the request, I passed the edit goal down to the editing step and split rules by role. A problem where fixed phrases were duplicated in range edits was fixed by passing the assembled previous result and the failure reason to the retry; this stage passed 98. I then checked request fulfillment and meaning preservation with a separate LLM evaluation and allowed at most one rewrite using the failure reason, which passed 104. Because the evaluator rejected valid drafts by also judging the generation JSON format, I separated the generation and evaluation prompts, and I fixed a preservation check that was too strict when a heading and formatting change were requested together. If a draft fails again, no edit is returned. AI drafts instructions and edits, code checks permissions and execution scope, and users decide publication and final application.

Scope and tradeoffs

Normalized average editing-evaluation time increased from 11.11s to 18.92s. I accepted the latency while limiting rewrites.

[Building user-defined Agent Skills \(Korean\)](#) ↗

[The two sides of LLM evaluation for reliability \(Korean\)](#) ↗

Cancelling an AI task and undoing what it already saved

An AI task saves documents, folders, wiki pages and skills several times between model calls. Stopping execution alone leaves part of a cancelled task behind, so I built a path that undoes already-committed changes in reverse order. Contributions left by other tasks survive, because recovery excludes only the target contribution from an append-only history.

2026.08 — 2026.09

Compensating recovery

Idempotency keys

Append-only history

PostgreSQL execution lock

Audit history

Background

An AI task runs across several stores and cannot be wrapped in one long transaction, so I had to decide what a cancellation should undo and how far it should reach.

Stores covered by
recovery

3

Recovery execution

**No model
calls**

Cancellation becomes
final

**On
confirmed
recovery**

Decisions and implementation

1 Deciding that stopping execution was not enough

Problem

An AI task saves documents, folders, wiki pages and skills several times between model calls, so stopping execution left part of a cancelled task in place. Execution spanning the core DB, the AI DB and object storage could not be wrapped in one long transaction.

Decision

I chose to undo already-committed changes too. Asking the model to perform the inverse was rejected because it cannot guarantee original values, approval scope or idempotency, and locking the whole service in one transaction was rejected because it cannot absorb long-running model, HTTP and object-storage work or its failures.

Implementation

The AI side derives reverse actions from the run record while the backend performs atomic recovery of business data. Agent tools store a pre-change snapshot with reverse-action arguments and an idempotency key tied to the original approval, and ordinary tasks record before and after values through each database's triggers inside the same transaction. Object storage persists the original and the intended result before writing.

Verification

Confirmed that recovery runs from the run record alone, with no model settings or calls.

Decisions and implementation (continued)

2 Blocking later writes and undoing children first

Problem

A handler still running after a cancellation would overwrite recovered values, and undoing a parent first left the changes its children had made behind.

Decision

Persist the cancellation first to block later writes, wait for the running handler to finish, then undo children first in reverse order.

Implementation

The agent turn and recovery share one PostgreSQL execution lock. The lock releases when a worker connection ends, so a leftover in-progress record cannot block recovery. Body recovery is saved as a new revision, so a late event cannot overwrite the source again.

Verification

Cancellation becomes final only once full recovery is confirmed; conflicts and unclear outcomes are left in a retryable state.

3 Recovering without erasing other tasks' contributions

Problem

A wiki concept page merges contributions from several documents, so overwriting it with an older snapshot also removed valid contributions added later by other documents.

Decision

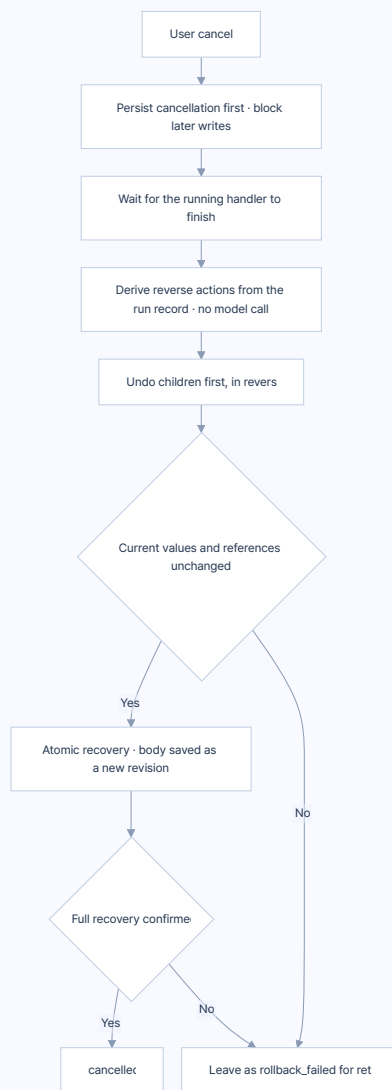
Removing just the relevant part from the finished result was rejected because which sentence came from which document cannot be told apart reliably, and restoring an entire older snapshot was rejected too. Instead the history is append-only and recovery excludes only the target contribution before reassembling.

Implementation

Operations, page revisions and per-document contributions are recorded append-only, and recovery restores or reassembles the page with the target contribution excluded. Pages and history are soft-deleted rather than removed, which preserves the audit trail.

Verification

Verified through revision and contribution hashes that state had not shifted during asynchronous recovery, and recorded the recovery itself as a new operation and revision.



A cancelled task's changes are undone while other tasks' contributions survive

Stores covered by recovery

3

core DB · AI DB · object storage

Recovery execution

No model calls

Reverse actions from the run record only

Cancellation becomes final

On confirmed recovery

Otherwise kept retryable

Content, names, parents and ordering that users see are restored, while version, audit and idempotency records are kept. Recovery runs from the run record alone, with no model settings or calls, and it checks current values and downstream references so it never overwrites another task's work. Cancellation is final only once full recovery is confirmed; conflicts and unclear outcomes stay retryable. For a feature where AI edits a user's documents, I settled what could be undone and what had to be recorded before building it.

Scope and tradeoffs

This case is about design and unit-level verification; I did not measure a recovery success rate under production traffic. Preserving per-source contributions from ingest onward costs storage. Without a snapshot of the remaining combination a page has to be reassembled, and older tasks carried no snapshot, so runs and queues had to be drained before deploying.

CASE 04

Fruition / PDF conversion · Data pipeline

PDF → Markdown: internal pass rate 45.17% → 89.89%

To use research papers as knowledge, PDF-to-Markdown conversion had to preserve table values and equations. I traced omissions introduced during AI repair and split processing paths by content type and original layout.

2026.07 — 2026.09

Open-source PDF conversion: AnyDoc, Docling

LLM API

sLLM (Qwen2.5 7B)

Python

LLM-as-a-Judge

Background

Using research papers as service knowledge required Markdown that preserved not only body text but also tables, equations and figures.

Existing Docling

45.17%

Improved pipeline

89.89%

Experiments and improvements

1 Selective repair and missed errors

Problem

Open-source PDF-to-Markdown conversion damaged table row/column relationships and equations. I needed a way to repair errors without processing everything again.

Experiment and result

Docling extracted the structure, and only damaged tables and equations were repaired with local Qwen sLLMs. Tables, where row/column positions matter, were handled mainly by the vision model (qwen2.5vl:7b); equations were drafted by the text model (qwen2.5:7b) and checked by the vision model only when needed. This restored more than applying both models to everything and cut sLLM processing time by about 42%. Keeping OCR and prompts unchanged and only re-cropping images with Docling's strict bboxes fixed two previously failing equations.

Evaluating repaired and untouched regions separately, 85 of the 96 sLLM-repaired regions passed, but 107 errors or omissions remained in untouched content. A separate experiment adding OCR comparison raised the pass rate from 80.00% to 97.53%, but also selected 50 correct regions for repair; the sum of processing-stage times was about 12 minutes 35 seconds.

Decision

Selecting repair targets missed errors, and extra checks to catch them added cost. I decided to test whether having the vision sLLM read every original page could avoid this dependence on damage detection.

Experiments and improvements (continued)

2 Table/equation errors and cost in full-page sLLM conversion

Problem

Selective repair missed errors. Adding OCR checks also flagged correct content, increasing evaluation time and cost.

Experiment and result

Full conversion of 25 research PDFs with the vision sLLM (qwen2.5vl:7b), which can check the original page directly, (415 pages) achieved a 91.92% content-preservation pass rate. However, only 58.84% of tables and equations fully matched in values and structure, and processing every page imposed a heavy processing load.

Decision

Rather than send all content through the same model path, I decided to use AI only where needed for body text and give tables and equations inputs that preserved their structure. I looked for fast text extraction with selective AI repair.

Experiments and improvements (continued)

3 Comparing text extraction, adopting AnyDoc, and moving to commercial APIs

Problem

Full-page sLLM processing still left table/equation errors and required sending all body text to the model. I needed fast text extraction alongside separate table/equation repair.

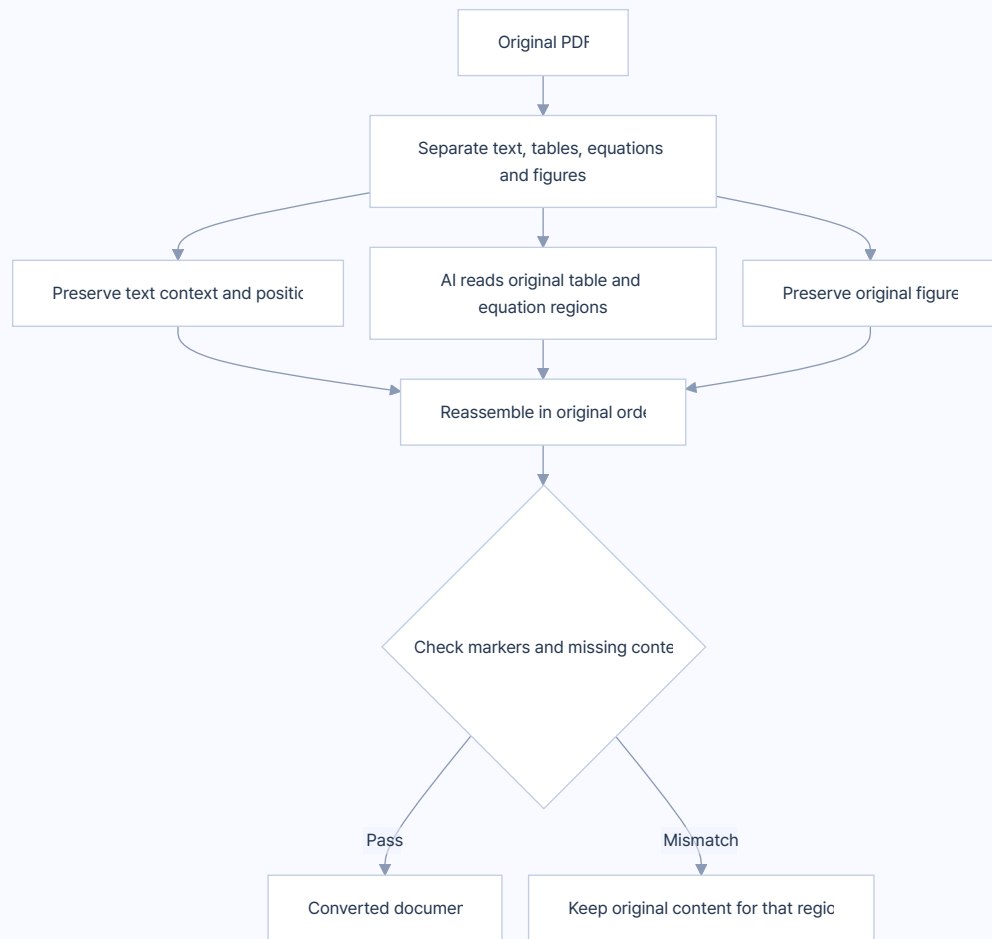
Experiment and result

I found AnyDoc on Threads and, after seeing its GitHub comparison with Docling, tested it on the same 4 PDFs (30 pages).

The recorded Docling run took 134.19s, while AnyDoc text extraction took 1.68s, about 80 times shorter. Docling's time also included OCR and table analysis. One AnyDoc document had garbled characters because the PDF lacked mappings from character codes to actual characters.

Decision

The processing scopes differed, so this was not an 80-fold improvement in complete conversion. Still, AnyDoc was useful for fast text extraction. I kept separate table/equation repair, switched body extraction to AnyDoc, used AI on garbled or misordered pages, and preserved original figures. With no early users yet, running GPU servers for an sLLM would cost more than it saved, so from this point I switched the repair model to commercial LLM APIs.



Pass rate against 30 original pages: 45.17% → 89.89%

Existing Docling

45.17%

30 pages total

Improved pipeline

89.89%

30 pages total

Split the same papers into 445 evaluation units covering body text, tables, equations and other content, then compared them with the originals. The existing Docling output and improved pipeline output were evaluated with method names hidden. The practical pass rate rose from 201 (45.17%) to 400 (89.89%), and the strict rate, which requires values and symbols to match exactly, rose from 193 (43.37%) to 393 (88.31%).

Scope and tradeoffs

Internal LLM evaluation of 4 PDFs (30 pages). Minor differences were allowed for text, headings and figures; tables and equations had to match in values, symbols and structure.

[Trying a small LLM for PDF → Markdown \(Korean\) ↗](#)

[Trying the open-source PDF converter AnyDoc \(Korean\) ↗](#)

CASE 05

Fruition / Kafka asynchronous processing · AI pipeline

Parallel document analysis: 282.11s → 73.89s

Sequential Wiki generation could make large documents slow to process and hurt the user experience. I measured worker scaling and resource use, then set different scaling policies for document analysis and embedding generation.

2026.09

AI pipeline

Kafka

PostgreSQL

Concurrency and memory

Background

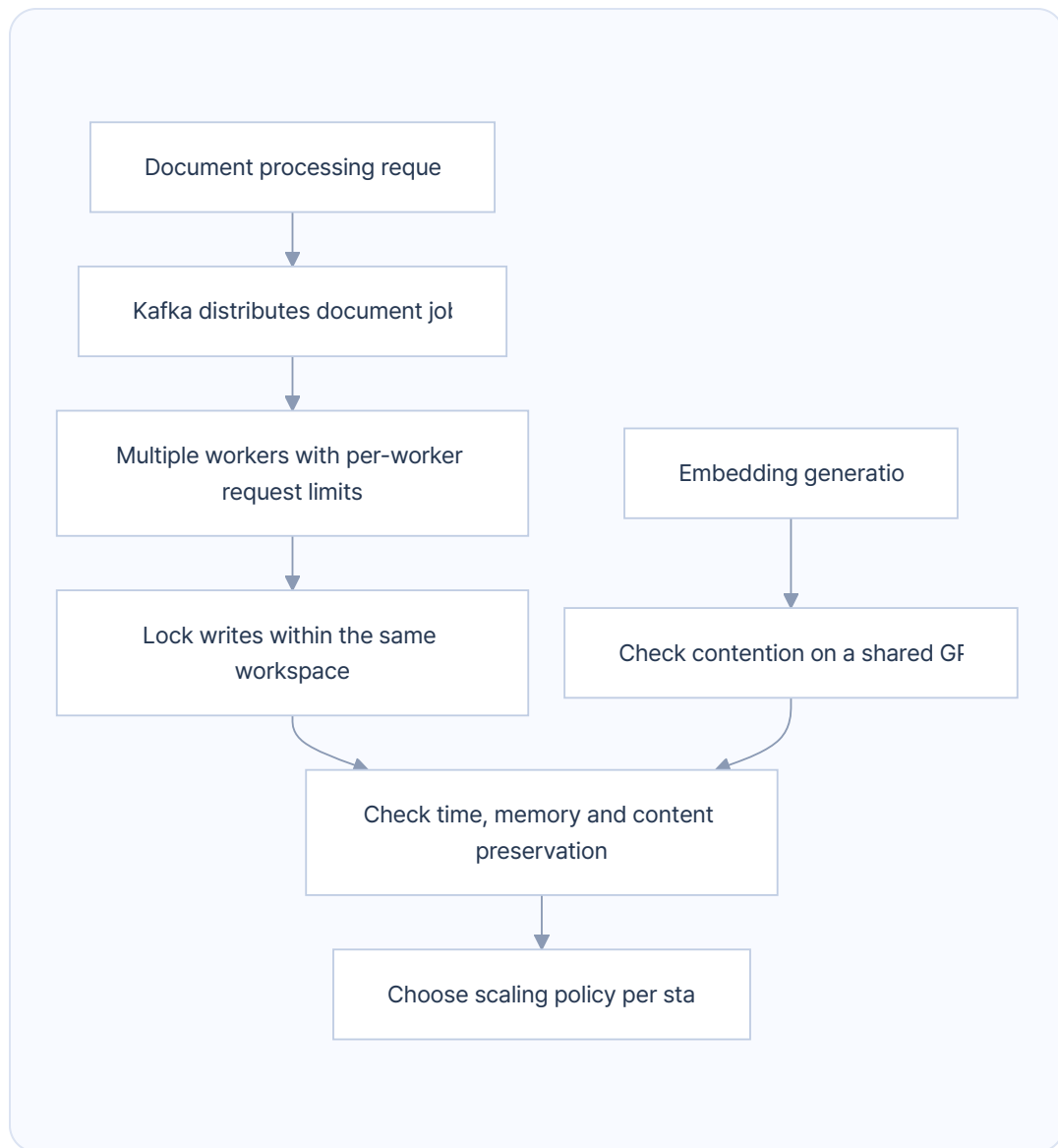
Sequential processing left later documents waiting for earlier external AI responses. Raising workers from 1 to 4 cut analysis time for 4 documents from 282.11s to 73.89s, so I tested whether more concurrency kept helping. Raising concurrent AI requests per worker from 2 to 4 slowed it to 95.92s, and running 2 embedding processes on the same GPU raised each from about 9.5s to 19–21s. Analysis waiting on external responses and embeddings sharing a device scale differently, so I set separate thresholds per stage.

1 worker

282.11s

4 workers

73.89sCombined process
memory**About 4×**



73.8% less analysis time for 4 documents · 300 queued jobs with zero loss

1 worker

282.11s

4 documents, 8 calls

4 workers

73.89s

Up to 8 concurrent API requests

Combined process memory

About 4×

351.7 → 1,401.0MiB

With 4 workers and 2 concurrent requests each, analysis time for 4 documents fell by about 73.8%. Combined process memory rose from 351.7MiB to 1,401.0MiB — about fourfold. Of 16 facts fixed in advance, 10, 14 and 11 were fully preserved with 1, 2 and 4 workers, so although each condition ran once on a small sample, I did not treat quality as identical. I rejected both raising concurrent requests further and parallelising embedding generation on the same GPU, since each made things slower. My initial assumption that more concurrency is always faster did not hold, and I began treating external-response waits and device contention as separate problems. Replaying saved AI responses for 100 documents under 3 queue configurations processed all 300 jobs without omissions or failures. I adopted a maximum of 4 workers: although memory increased about fourfold, I judged that the AI service's MSA deployment on EKS could distribute the load.

Scope and tradeoffs

One analysis run per configuration on the same documents. These are not end-to-end timings including storage and post-processing.

Coding agent loop: 16 MSA migration PRs merged in 5 days

I delegated moving the AI features into the MSA structure to coding agents, while I set the work plan and review criteria and decided what to merge. Agents ran in parallel in a separate workspace per branch, and a coordinator agent managed order and priority.

2026.08

Coding agents

Claude Code

MSA

Code review

Background

With about a week left before the MSA migration deadline, moving the AI features into the new service structure had split into several streams. Long single sessions lost track of the plan as context was summarized.

PRs merged

16

Required fixes before merge

13

Agents per branch

2

Loop design and decisions

1 Building an agent loop with separate roles

Problem

Giving one session all the work blurred the plan and criteria as context was summarized.

Experiment and result

Fixed the work plan and criteria in documents and split roles into implementation, review and integration review. Each branch had its own workspace with two agents running in parallel, and a coordinator agent judged dependencies and priority between tasks.

Decision

Sessions could change and still continue from the same documents, so splitting long work across agents did not scatter its direction.

2 PR timeline: 8/9 → 8/13

Problem

Tasks depended on each other, so opening everything at once risked conflicts and rework.

Experiment and result

Opened #156–#159 on 8/9 and merged them on 8/10, ran #167, #169, #170 and others in parallel on 8/11, then fixed integration defects in #175 on 8/13. 16 PRs merged over 5 days.

Decision

Opening the next batch only after prerequisite work merged kept conflicts down while still running in parallel.

Loop design and decisions (continued)

3 Pre-merge review and merge decisions

Problem

Merging agent-made changes as-is risked bringing defects in with them.

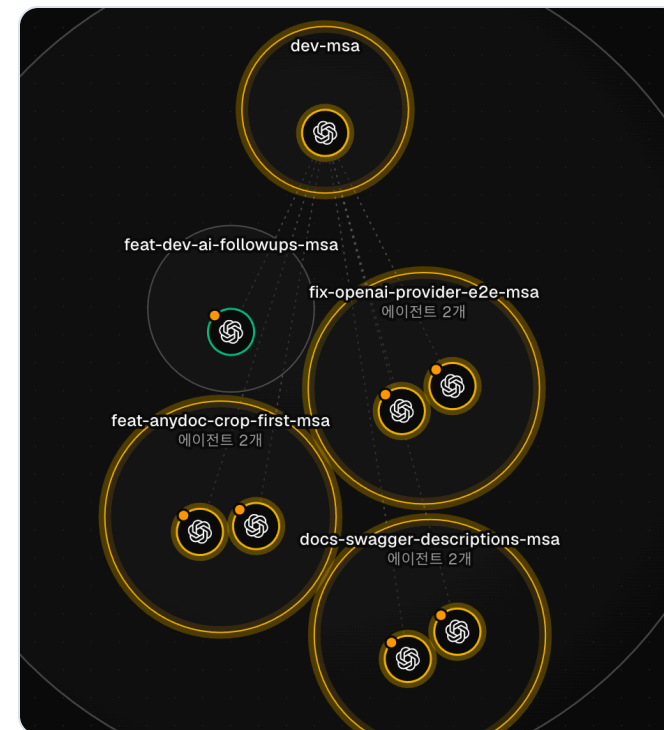
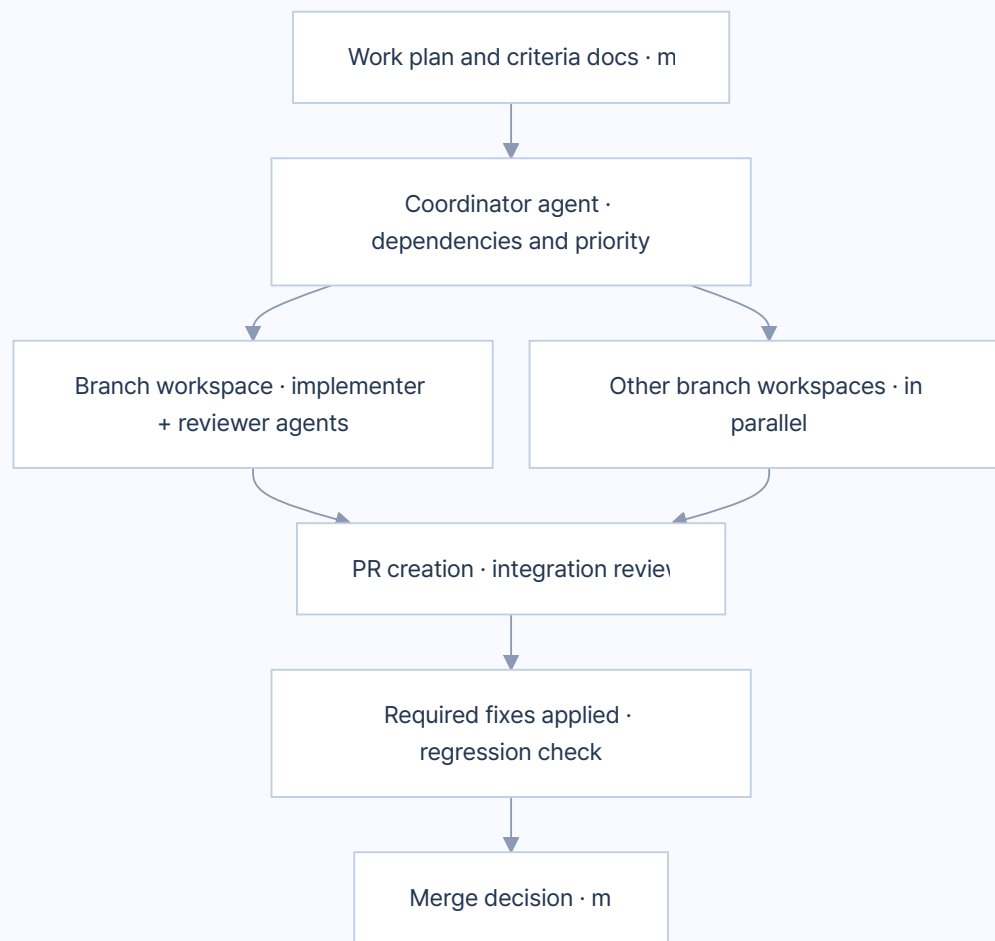
Experiment and result

Merged only after the 13 required fixes from review were applied and regressions checked. For example, a defect where restoring a document returned its previous body was fixed before merge.

Decision

Instead of writing the code myself, I made my role defining the pass criteria and deciding merges by them.

Improved processing flow



I set the plan and criteria; agents with separate implementer and reviewer roles worked in parallel per branch.

16 PRs merged in 5 days · 13 required fixes applied before merge

Merged 16 PRs over 5 days, from August 9 to 13. The 13 required fixes raised in pre-merge review were applied and checked for regressions. One defect the review caught: restoring a document returned its previous body. The coding agents did the implementation; I designed the loop, wrote the work plan and criteria documents, reviewed results and made merge decisions.

Scope and tradeoffs

I judged merges by criteria documents and review results rather than writing the code myself, so defects outside the review criteria can slip through. Refining the criteria documents remains ongoing work.

PRs merged

16

2026.08.09 – 08.13

Required fixes before merge

13

From review

Agents per branch

2

Implement · review

Pilltip

A personalized AI medication service that explains drug information in plain language and helps users manage their own and their family’s medications and schedules.

How it started

Started as a Pusan National University capstone project. Drug information is scattered, so it is hard to check whether a given medicine is safe for your current health condition. We set out to make that easy.

[GitHub repository ↗](#)

Team and my role

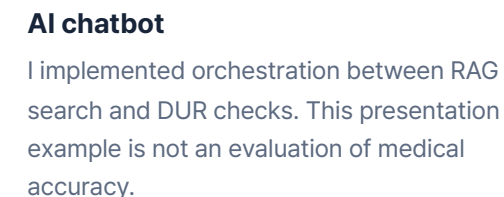
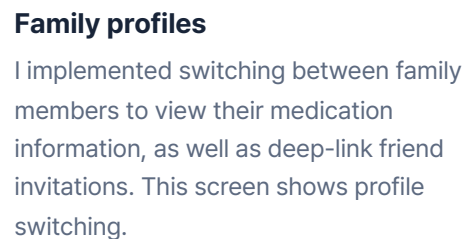
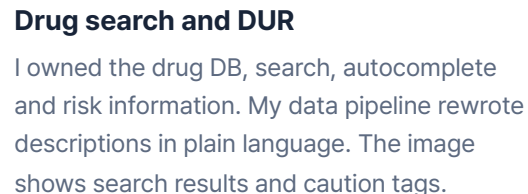
MY ROLE

Backend & AI

Drug and supplement DB, search and autocomplete, reminders, risk and caution information, data preparation, AI chatbot

- 01 Backend and AI development
- 02 Drug and supplement modeling, search, autocomplete and risk information
- 03 Medication reminders and logs
- 04 Friends and family
- 05 Data pipeline: about \$200 estimated, \$11.18 actually spent
- 06 RAG-based AI chatbot
- 07 Extended support to health functional foods after exhibition feedback

Android, 1	The Kotlin-based Android app
Backend & security, 1	User DB, login, questionnaires, AES-GCM encryption
Design	Designer



CASE 07

Pilltip / Data pipeline · GPT Batch API · PySpark

Drug data pipeline: \$200 estimated → \$11.18 spent

Built a data preparation pipeline to rewrite difficult drug information in plain language. Split the text into sentence blocks, converted each unique sentence once and mapped results back to each drug, for \$11.18 in actual API spending. In 2026 I reproduced the same preparation in PySpark to measure the effect of each stage.

2025.07 · reproduced 2026.10

Data pipeline

GPT Batch API

PySpark

MinHashLSH

Background

Built a feature to rewrite about 44,000 drug records into explanations users could understand.

Estimated cost for
original text

**About
\$200**

Actual API spending

\$11.18

Exact deduplication

-94.1%

Problem analysis and implementation

1 Removing exact duplicate sentences

Problem

Converting about 44,000 records totaling 1GB was estimated to cost about \$200 in API usage. I needed to cover all drugs within budget.

Experiment and result

Examined 100 records and found about 70% duplicate content. Used this sample and a work plan to secure 7 days for preprocessing. Split text into sentence blocks, merged exact matches and preserved links from each drug to its blocks.

Decision

Convert repeated sentences once and reuse them rather than cover fewer drugs.

2 GPT Batch API conversion and mapping results back to drugs

Problem

Deduplication reduced input, but I still needed to minimize model costs.

Experiment and result

The work did not require immediate responses during service use, so batch waiting was acceptable. Converted unique sentences with GPT Batch API, then reused the preserved drug-to-block mappings to attach results to each drug. Actual API spending was \$11.18 versus about \$200 estimated for all original text.

Decision

Combined deduplication and batch conversion to deliver the feature without reducing drug coverage.

Problem analysis and implementation (continued)

3 2026 PySpark reproduction: measuring ending normalization and similar-sentence merging

Problem

Part of the 2025 code and the raw files no longer existed, so the effect of stages after exact deduplication could not be shown in numbers.

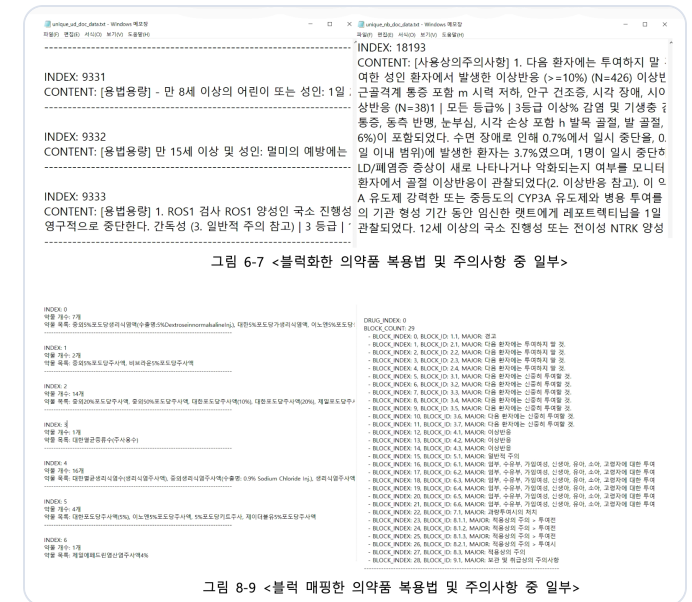
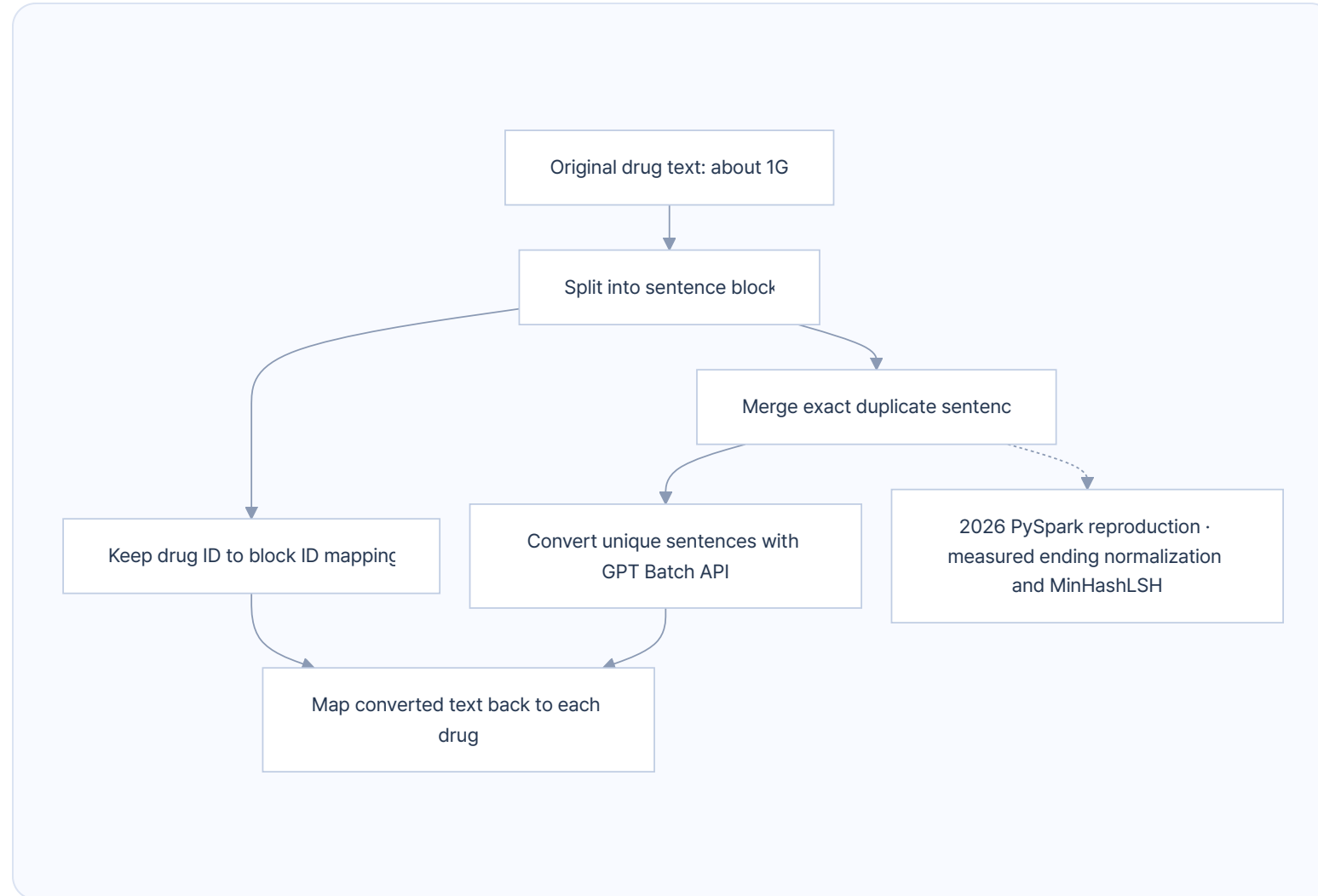
Experiment and result

Re-collected 42,714 products from the MFDS public API, split them into 3,299,602 blocks following the XML structure, and reproduced normalization and exact deduplication in PySpark. Then applied an ending-normalization dictionary and MinHashLSH (character 3-grams, Jaccard distance), comparing reduction and merged samples at thresholds 0.1, 0.2 and 0.3.

Decision

At 0.3, text shrank 18.78% but sentences about different conditions (Lapp vs. total lactase deficiency) merged, and at 0.2 some precaution sentences were lost. Wrong merges are costly for medical information, so I chose 0.1 (a further 6.69%) with guards that merge only when numbers and applicability conditions match.

Improved processing flow



Converted each repeated sentence once and kept per-drug mappings to attach results. In 2026 the same process was reproduced in PySpark and measured.

Actual API spend \$11.18 · PySpark reproduction cut text by 94.1%

Completed conversion for \$11.18 in actual API spending versus about \$200 estimated for all original text. Reusing converted unique sentences kept the feature within budget without reducing the set of drugs covered. Part of the 2025 preparation code no longer existed, so in 2026 I re-collected 42,714 products (960MB of plain text) from the MFDS public API and reproduced the same preparation in PySpark. 3,236,626 blocks reduced to 176,648 unique blocks, cutting text from 744MB to 44.1MB (−94.1%). An ending-normalization dictionary then removed 0.78%, and MinHashLSH similar-sentence merging a further 6.69%.

Scope and tradeoffs

About \$200 was the upfront estimate; \$11.18 was actual spending. The 2026 figures come from re-collected data, not the 2025 dataset. The reproduction code was written with AI coding tools; I designed the experiment and chose the thresholds.

Estimated cost for original text

About \$200

Estimate

Actual API spending

\$11.18

2025 conversion

Exact deduplication

−94.1%

2026 PySpark reproduction · 744MB → 44.1MB

CASE 08

Pilltip / AI Agent · Sensitive data boundaries

Medication chatbot: keeping health profiles out of external AI inputs

Built a medication chatbot using RAG and AI orchestration for app integration. Separated symptom-based drug discovery from health-profile decisions, using the model for language understanding and code to control sensitive profile use.

2025.09

Spring AI

Weaviate

RAG

AI Orchestration

Background

Built a chatbot that finds drug information from natural-language symptoms and personalizes it using health profiles.

Design and implementation

1 Semantic search connecting symptoms to drug indications

Problem

Everyday symptom descriptions and drug indications use different words for the same meaning. Related information needed to be found without exact keyword matches.

Decision

Use semantic search to connect symptoms and indications, with separate processing paths for the information each question needs.

Implementation

Stored drug indications in Weaviate for semantic symptom search. Built paths for drug discovery, medication risks and dosage information according to the question.

Verification

Implemented natural-language symptom search and routing to the appropriate information-processing path.

2 Internal health-profile processing, separate from external AI inputs

Problem

Personalization required pregnancy status, medications and underlying conditions, but sending those profiles to an external LLM risked exposing sensitive information.

Decision

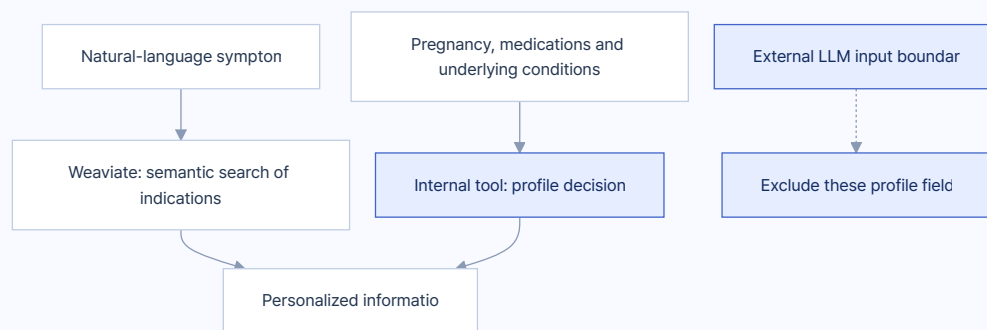
At first I considered sending the health profile to the external AI with sensitive content filtered out. But there were too many expressions and cases to filter, and I judged that code could not cover them all. So I separated understanding symptom language from deciding profile conditions, and kept profile decisions in internal code where inputs and rules can be controlled directly.

Implementation

Excluded pregnancy, medication and underlying-condition fields from external LLM inputs. Connected internal profile decisions to symptom search results to provide personalized information.

Verification

Separated processing paths so those health-profile fields stayed out of external LLM inputs while remaining available for internal personalization.



Separating symptom search from internal health-profile decisions

Enabled natural-language drug discovery while excluding pregnancy status, current medications and underlying conditions from external LLM inputs. Explicitly separated information sent to AI from information evaluated by internal code. In a domain that handles sensitive data, I settled the boundary between what the model may judge and what code stays responsible for before writing the implementation.

Scope and tradeoffs

Profile-rule changes require code updates. Even so, limiting how much sensitive health data reaches external models came first, so this decision stays in code rather than with the AI. It removes specific profile fields from external inputs; it does not measure blocking across every possible path.

Beyond project development

Rhwp · Fixing HWPX save errors

Gave Claude Code a work plan and contribution rules to automate the flow from issue analysis to PR; 7 PRs merged, including the textFlow preservation fix (#1213), and listed as a contributor.

[GitHub repository](#) ↗

APPTIVE · Backend mentoring

Used my experience as a mentee to improve teaching. Led 6 mentoring and code-review sessions for 12 mentees.

945-day Baekjoon problem-solving streak

Aiming for one problem a day, I reached a 945-day streak on solved.ac. 1,659 problems solved, solved.ac Platinum IV.

[Solutions repository](#) ↗

Project reflections

ARCHITECTURE / Data ownership

Moving to MSA and learning from feedback

Shifted from splitting by feature to defining data ownership. Documented both the benefits of asynchronous processing and the added operational burden.

PLANNING / Defining user problems

Reflections on design thinking

Revisited feature-first planning and brought together the team's difficulties finding documents to define the problem we would solve together.

[More blog posts ↗](#)

Jaehyeong Kim

AI Application Developer · Backend Engineer

kkuldangi2@gmail.com

martinel2.github.io

github.com/Martinel2

I owned the whole AI side of an LLM workspace, designing its RAG, agent and evaluation layers and building and verifying them with AI coding tools.

I weigh accuracy, cost and response time against each other, confirm the trade-off with an evaluation, and then choose the implementation that fits the service.